TensorGuard: Static Refinement Verification for PyTorch Tensor Programs

TensorGuard Authors

Abstract

TensorGuard is a static verifier for PyTorch models that turns tensor-program well-formedness into refinement constraints. It checks shape, device, dtype, train/eval phase, stride, permutation, and gradient-flow facts before training or export begins, while preserving an explicit three-valued soundness contract: when the verifier cannot prove a claim inside its supported fragment it reports UNKNOWN rather than silently passing. The implementation combines a source frontend, a `torch.fx`/export-oriented graph frontend, a reduced product of tensor abstract domains, Z3-backed safety checks, CEGAR-style predicate discovery, differential conformance tests against live PyTorch, Lean proof artifacts for core rules, and reproducible empirical harnesses.

The repository is not merely a prototype pass over toy models. It contains operator transfer functions for everyday PyTorch programs, frontends for real `nn.Module` code, integration hooks for CLIs, GitHub Actions, SARIF, watch mode, Jupyter, decorators, framework adapters, deployment gates, and artifact scripts that regenerate precision/recall, latency, ablation, coverage, conformance, fuzzing, mutation, and benchmark evidence. This paper describes the system as shipped in the repository, with emphasis on the engineering and evaluation choices that make TensorGuard a credible research artifact and a useful developer tool.

Problem and Claim

Most deep-learning correctness tools see tensor bugs too late. A model can import successfully, type-check as Python, instantiate its modules, and even reach a training loop before a wrong channel count, a bad attention projection, an accidental CPU buffer, a mismatched dtype, or a train/eval phase dependency raises a runtime exception. Worse, some failures are silent: a detached tensor can sever gradient flow, a broadcast can mask an intended batch axis, or an export pipeline can accept a graph whose serving contract no longer matches the model author's intent.

TensorGuard attacks these failures as static contract violations. A tensor value is modeled with refinement information such as [v : Tensor Shape(v)=(B,C,H,W) Device(v)=cpu DType(v)=float32 .] The verifier propagates these facts through constructors, module calls, functional operators, reshapes, broadcasts, indexing, attention blocks, normalization layers, sparse layouts, distributed wrappers, and deployment gates. At each operation it asks whether the preconditions of the transfer function are entailed by the current abstract state. Concrete contradictions become high-confidence bugs. Unknown or out-of-fragment regions become abstentions with reasons, not fake successes.

The central claim of the repository is practical: a tensor verifier can be soundly conservative and still catch real bugs early across modern PyTorch workflows. The codebase supports that claim with four pillars.

• A precise public contract that separates SAFE, UNSAFE, and UNKNOWN outcomes, with a strict ``sound'' mode for CI.

• A broad operator and frontend implementation that handles real modules, not only a calculus in a paper.

• A reproducible evaluation suite with real-bug corpora, differential testing, mutation tests, benchmark scripts, ablations, dashboards, and baseline comparisons.

• A formalization and conformance story: Lean artifacts for core transfer rules, Z3 encodings for symbolic constraints, and live PyTorch oracles for operator behavior.

This is a tool paper: the interesting contribution is the system boundary, not a single transfer function. The repository demonstrates how a static analysis tool can be packaged so that reviewers, users, and future contributors can inspect the guarantee, reproduce the headline evidence, and extend the coverage without weakening the trust story.

Architecture

TensorGuard is organized as a pipeline from Python source or captured graphs to a multi-domain safety decision. The source path parses `nn.Module` classes, extracts constructor parameters and layer definitions, walks `forward`, and emits a computation graph. The graph path consumes `torch.fx`, `torch.export`, and integration-specific traces where available. Both paths feed a common verifier built around symbolic tensor states.

Each state maps tensor names to abstract facts. Shape facts are tuples of concrete integers or symbolic names; device facts range over CPU and CUDA devices; phase facts track train/eval context; dtype facts model promotion and casts; gradient facts track whether a value can carry gradient to downstream parameters. Several later steps add stride and permutation information so viewability and layout-sensitive operators can be checked without collapsing to opaque shapes. These domains form a reduced product: a violation in one domain can be explained with facts from the others, while disabled domains are gated out of the solver rather than filtered only at the end.

The pipeline has four visible outputs. First, it returns structured bugs with locations, severities, confidence values, and optional counterexamples. Second, it returns a verdict: SAFE, UNSAFE, or UNKNOWN. Third, it can emit developer-facing diagnostics, including source snippets, inferred-vs-expected shapes, explanations, and autofix suggestions for mechanical cases. Fourth, it can emit machine-readable reporters: JSON, JUnit XML, SARIF 2.1.0, GitHub annotations, coverage artifacts, dashboards, and certificates.

Frontends

The source frontend is deliberately useful on unannotated Python. It recognizes `nn.Linear`, convolution families, pooling, normalization, attention, activation layers, containers, helper methods, subclassing patterns, and functional calls such as `F.linear`, `F.interpolate`, and `F.scaled _dot _product _attention`. It records scalar constructor attributes such as head counts and hidden sizes, so common model code like `x.view(B, T, self.num _heads, D)` can be checked against the same constants used by the layers.

The graph frontends target execution-oriented workflows. `torch.fx` support traces real modules and preserves enough source mapping for diagnostics. `torch.export` and Dynamo-oriented paths let TensorGuard run as a pre-pass for export and compile pipelines. The repository also includes integration points for guarded `torch.compile`, AOTInductor packaging, ONNX export, GGUF/llama.cpp-oriented checks, framework hooks, and deployment request/response schemas. These paths are important because many production tensor bugs appear only when a model is compiled, exported, quantized, sharded, or served.

Verifier

The verifier performs stepwise symbolic propagation. For each computation step it updates the tensor state, emits safety conditions, and asks Z3 whether a violation is forced. Concrete mismatches, such as a `Linear` layer expecting 512 features but receiving 256, are immediate. Symbolic mismatches are handled conservatively: if the verifier can prove incompatibility under the known assumptions it reports a bug; if the result depends on an unconstrained symbol, it abstains or keeps the dimension symbolic.

The same engine also supports CEGAR. When the current abstraction is too weak, TensorGuard proposes additional shape predicates, reruns the proof attempt, and records the discovered predicates as metadata. If the predicates jointly imply an impossible input contract, the conflict is promoted to a real `cegar _refined _contract` bug rather than hidden as an internal note. The repository includes convergence measurements and Lean-checked statements for the predicate-bound argument.

Soundness Modes

The user-facing contract is intentionally explicit. In `sound` mode, TensorGuard reports SAFE only when the program remains inside the verifiable fragment and all relevant transfer functions are at an acceptable confidence level. Unsupported layers, opaque data-dependent control flow, or heuristic operators make the verdict UNKNOWN unless a concrete bug is found. In `balanced` mode the tool remains useful for day-to-day development by allowing more best-effort reasoning, while still surfacing abstentions. In `heuristic` mode users can opt into permissive exploratory checks. The mode changes the verdict boundary; it does not hide discovered bugs.

Tensor Semantics and Transfer Functions

A transfer function is the contract for one operator family. It consumes input abstract facts and produces output facts or a violation. TensorGuard tags each operator as complete, sound, or heuristic. Complete rules are exact shape-preserving families such as pointwise activations. Sound rules enforce well-defined structural contracts, even if the implementation abstains on hard symbolic cases. Heuristic rules are used when output rank or size depends on runtime values, or when the rule intentionally approximates a large behavior surface.

The repository includes transfer functions and tests for a wide range of PyTorch constructs. The core structural set covers matrix multiplication, batched matrix multiplication, linear layers, convolution and transpose convolution families, pooling, flatten, reshape/view with product reasoning, broadcasting, reductions, cat/stack, transpose/permute, squeeze/unsqueeze, embedding, dtype casts, device moves, and stochastic tensor factories whose shapes are seed-independent. Extended coverage includes einsum parsing, advanced indexing, gather/scatter, index select, masked select, take-along-dim, sort/topk families, sparse COO/CSR/CSC/BSR/BSC layout invariants, FFT/complex view contracts, named tensor alignment, vmap/functorch batch dimensions, distribution batch/event/log-prob shapes, grid sampling, affine grids, multi-head attention, scaled dot-product attention, LoRA adapter compatibility, training-loop checks, quantization/export safety, distributed sharding, tensor parallel attention variants, and deployment schemas.

Exact Partition Operators

The latest completed implementation round strengthens `torch.chunk` and `torch.split`. These operators look simple, but their PyTorch semantics matter in downstream checks. `torch.chunk(x, chunks)` may return fewer than `chunks` outputs when the split dimension is positive and smaller than requested. For example, chunking a length-5 dimension into 8 chunks returns five length-1 tensors, not eight tensors with empty tails. A zero-length dimension is special: chunking length 0 into 3 chunks returns three empty tensors. Integer `torch.split` produces fixed-size chunks with a possibly smaller final chunk; section-list `split` requires the section sizes to sum exactly to the dimension and permits zero-sized sections.

TensorGuard now uses shared PyTorch-faithful calculators for these rules. Tuple unpacking records one graph step per output element, so `a,b,c = x.chunk(3, dim=-1)` gives `c` the actual final-chunk width. This is essential for catching a downstream `nn.Linear` that expects the first chunk's width but receives the smaller final chunk. The same rule handles method and functional forms, negative dimensions, empty sections, and concrete unpack-count mismatches. When the partition size or output count depends on a symbolic dimension, the split axis is replaced with a fresh symbolic dimension while non-split dimensions are preserved. That preserves useful downstream checks without inventing a false concrete chunk size.

Examples of Transfer Precision

Reshape and view rules reason about element products and the `-1` inference sentinel. They reject concrete product mismatches, infer the missing dimension when possible, and preserve copied symbolic dimensions through `x.size(i)` and `x.shape[i]` aliases. Broadcasting implements NumPy/PyTorch right-aligned semantics, including singleton expansion and symbolic abstention. Convolution rules compute spatial output sizes from kernel, stride, padding, dilation, groups, and transpose output padding when these parameters are static. Normalization rules check channel or normalized shape contracts, while phase annotations make train/eval-sensitive modules diagnostic instead of invisible.

Attention rules are handled at multiple layers. SDPA verifies query/key embedding compatibility and key/value source-length compatibility while preserving the output value dimension. Multi-head attention checks packed and unpacked query/key/value forms, `kdim`/`vdim`, batch-first layout, attention masks, key-padding masks, per-head weight shapes, and valid symbolic cases. Tests replay these contracts against real PyTorch modules and functional calls.

Data-dependent operators are treated honestly. `masked _select`, `nonzero`, single-argument `where`, and boolean-mask indexing can produce extents that depend on runtime tensor values. TensorGuard now first checks the static part of the contract - mask broadcastability or boolean-mask prefix shape - and then emits symbolic extents plus explicit UNKNOWN reasons instead of guessing a concrete selected length. This design is repeated throughout the codebase: useful exactness where the contract is shape-determined, conservative abstention where it is not.

Developer Experience

The repository presents TensorGuard as a tool developers can actually adopt. The CLI supports zero-flag verification on common `nn.Module` files, high- confidence CI gating, domain flags such as `--no-device-check`, and soundness modes. Diagnostics are source-mapped and explain why an operation is unsafe, not just that Z3 found a model. The `--explain` path reconstructs the inference chain from inputs through transformations to the failing step. The `--fix` path suggests mechanical repairs for supported errors such as wrong `Linear.in _features` or convolution channel counts.

Adoption surfaces include a GitHub Action, SARIF for code scanning, JSON and JUnit reporters, GitHub PR annotations, pre-commit integration, nox/tox hooks, a pytest plugin, a

watch mode, Jupyter/IPython cell checks, a `@tensorguard.checked` decorator, a repository config file, framework hooks for higher-level trainers, and a public API with type stubs. The packaging story includes reproducible wheels, a pinned Z3 range, pipx usage, Docker, conda-forge metadata, and a documented security policy for untrusted model files.

The diagnostic design is intentionally evidence-oriented. A bug report can carry an expected shape, actual shape, line and column, a counterexample, the domain that produced the violation, and the chain of operations that made the state refutable. When a domain is disabled, the solver encoders for that domain emit no constraints, so the flag removes solver work instead of merely filtering the final verdict. This matters for ablation experiments and for users who need predictable CI behavior.

Empirical Methodology

TensorGuard ships with a broad evaluation harness. The artifact is organized around questions a reviewer or user would ask.

Does it catch real bugs? Does it avoid false positives on clean models? Does it agree with PyTorch on operator semantics? Does it scale? Which abstract domains matter? How does it compare to baselines? Can the entire claim set be regenerated from committed scripts?

The repository answers these questions through several benchlines and experiments rather than a single benchmark number. The frozen ground-truth corpus in `real _benchmarks/` contains clean and buggy PyTorch modules with content hashes and labels. The GitHub-mined dataset under `experiments _v5/github _bug _mining/` records real public issue and PR signals associated with PyTorch runtime error signatures. Fuzzing harnesses generate valid modules and mutation harnesses inject shape, device, and dtype faults. Differential conformance tests cross-check transfer functions against the live torch dispatcher. Baseline comparison scripts evaluate PyTea and runtime-tool alternatives. Dashboards and JSON artifacts track coverage, precision/recall, latency, ablations, false positives, and statistical rigor.

|p0.25|p0.68|

Evidence family and Repository artifact and purpose

Ground-truth corpus and `real _benchmarks/`, `tests/test _real _benchmarks.py`. Frozen clean/buggy modules with a manifest, hashes, labels, and loader checks.

GitHub-mined bugs and `experiments _v5/github _bug _mining/`. Offline dataset plus miner for public shape/device bug candidates from issue and PR history.

Precision/recall and `tests/test _precision _recall.py`, `tests/test _hard _recall.py`, `experiments/`. Tracks caught bugs, clean-model behavior, and high-confidence safety.

False-positive stress and `tests/test _fp _stress.py`, `tests/test _sound _mode _fp.py`. Exercises clean real-model patterns and sound-mode no-false-positive posture.

Differential dispatch and `tests/test _differential _dispatcher.py`, `tests/test _conformance _oracle.py`, `tests/test _propagator _contracts _runtime.py`. Compares transfer outputs to live PyTorch behavior.

Fuzzing and mutation and `tests/test _diff _fuzz.py`, `tests/test _neg _fuzz.py`, `tests/test _hypothesis _module _ast.py`, `tests/test _mutation _clean _models.py`. Searches for disagreements and injected bugs.

Operator coverage and `operator _confidence _table.json`, `tests/test _operator _coverage.py`, `tests/test _operator _coverage _gate.py`. Tracks supported operators, confidence tags, and release gates.

Latency and scaling and `tests/test _cost _benchmarks.py`, `tests/test _latency _budgets.py`, `tests/test _scaling _study.py`. Measures model-size and solver-cost behavior.

Baselines and `tests/test _baselines.py`, `tests/test _baseline _head _to _head.py`, `tests/test _headtohead _n15.py`. Compares against PyTea, runtime checks, and type-oriented alternatives where applicable.

Ablations and `tests/test _domain _ablation.py`, `tests/test _reduced _product _ablation.py`, `tests/test _cegar _depth _ablation.py`, `tests/test _verified _reduction _diff.py`. Measures contribution of domains and refinement depth.

Dashboards and `tests/test _dashboard.py`, `tests/test _build _dashboard.py`, `tests/test _leaderboard.py`. Regenerates static result views for reviewers and contributors.

Statistical rigor and `tests/test _statistical _power.py`, `tests/test _significance.py`, `tests/test _corrected _meta _analysis.py`. Documents effect sizes, power, and aggregate evidence.

Evaluation Highlights

The strongest evidence is not one number; it is the agreement between many small, independently checkable claims. TensorGuard's benchmark suite exercises real failure modes: wrong `Linear` feature counts after flattening, malformed convolution channels, bad Q/K/V dimensions, attention mask errors, invalid reshape products, device mismatches between buffers and inputs, dtype divergences, gradient detach bugs, invalid sparse compressed layouts, named tensor alignment failures, incompatible distribution batch/event shapes, export contracts, quantization observer placement, and distributed sharding mistakes.

The conformance tests are especially important. A static rule is only useful if it matches the runtime library. The repository therefore includes tests that construct real tensors, run the corresponding PyTorch operator, and compare the resulting shape or runtime exception to TensorGuard's predicted contract. This pattern appears for cat, einsum, reshape, broadcasting, convolution, attention, grid sampling, sparse layouts, complex FFT views, named tensors, distributions, vmap, and now chunk/split partitioning. When PyTorch's edge-case behavior is surprising, as with `torch.chunk` returning fewer tensors than requested, the static rule follows PyTorch rather than a simplified textbook semantics.

The ablation suite makes the multi-domain design measurable. Shape alone catches the most common architectural errors, but device and gradient domains catch bugs that shape-only tools miss. Dtype reasoning catches mixed-precision and cast-related failures. Phase information explains train/eval-sensitive patterns. The reduced product is valuable because real failures often cross domain boundaries: a compile/export path may require a shape, dtype, device, and layout combination, not merely one axis of correctness.

Latency and scalability scripts ensure the verifier remains useful. Solver queries are cached or avoided when a transfer can be decided syntactically. Constraint simplification performs constant folding, common-subexpression sharing, divisibility normalization, and short-circuiting. Per-module verification can run in parallel, and incremental analysis reuses results when only part of a model changes. Timeout and anytime modes return sound partial results instead of claiming full coverage after a budget expires.

Formal and Trust Story

TensorGuard does not ask users to trust only implementation tests. The repository contains Lean artifacts for several core rules and proof obligations, including CEGAR convergence bounds,

infeasible-contract abstention, fragment modes, device/dtype transfer soundness, SMT encoding faithfulness, cross-domain composition, and operator-specific rules such as cat and reshape families. The Lean audit checks theorem statements for sorry-free status under the trusted kernel axioms used by the environment.

The formalization is intentionally scoped. It does not claim to prove all of PyTorch. Instead, it proves the algebraic cores that the implementation relies on and pairs them with runtime conformance tests for library behavior. For a research artifact, this is a practical trust split: Lean validates the abstract rule, Z3 discharges symbolic conditions in the implementation, and PyTorch oracle tests validate that the rule corresponds to the library release being tested.

The soundness contract documents the remaining boundary. Unsupported or heuristic operators are not silently certified. Opaque layers and data-dependent control flow produce UNKNOWN in strict modes. Symbolic sizes are preserved instead of guessed. This is visible in the code. For chunk/split, for example, concrete dimensions produce exact per-output sizes and concrete unpack-count errors; symbolic split dimensions produce fresh symbolic split axes, preventing false claims about a final chunk size.

Implementation Surfaces

The system is intentionally broad because tensor bugs are not confined to a single frontend. The repository includes modules for source analysis, model checking, graph compilation, torch integration, export extraction, Dynamo gap analysis, framework hooks, sparse and complex verification, SDPA and MHA verification, distribution checks, named tensor checks, vmap checks, quant/export checks, distributed verification, LoRA verification, training-loop checks, reporters, SARIF, baselines, dashboards, proof certificates, and API stability.

|p0.24|p0.69|

Surface and Representative capability

`src/api.py` and Public `analyze`, `verify _architecture`, and `verify _module` entry points with soundness modes and domain flags.

`src/model _checker.py` and AST graph extraction, symbolic state propagation, Z3 safety checks, device, phase, gradient, dtype, and shape transitions.

`src/tensor _shapes.py` and Shape algebra, transfer helpers, AST shape analyzer, reshape/broadcast/cat and chunk/split shape computation.

`src/fx _extractor.py` and `src/export _extractor.py` and Tracing and export-oriented frontends for real modules and deployment flows.

`src/torch _integration.py` and Guarded compile/export/package pre-passes that surface TensorGuard violations before compiler or exporter failures.

`src/mha _verify.py`, `src/sdpa _verify.py` and Attention-family contracts for q/k/v, masks, head counts, and output shapes.

`src/sparse _verify.py`, `src/complex _verify.py` and Sparse layout and complex/FFT shape-dtype contracts.

`src/distributions _verify.py`, `src/named _tensor _verify.py`, `src/vmap _verify.py` and High-value library contracts beyond ordinary dense layers.

`src/quant _export _checks.py`, `src/distributed _verification.py`, `src/lora _verification.py` and Deployment, distributed training, and adapter-compatibility gates.

`src/reporters.py`, `src/sarif _codescan.py`, `src/github _action.py` and Machine-readable and platform-native reporting.

`lean/TensorGuard/*.lean` and Machine-checked statements for selected transfer rules, mode boundaries, encoding faithfulness, and CEGAR reasoning.

Artifact Reproducibility

The artifact is designed to be regenerated. The `make reproduce` harness and reproducibility scripts update headline results, numeric claim audits, benchmark deltas, scaling curves, CEGAR measurements, and paper evidence indices. Tests pin committed JSON artifacts against freshly generated outputs. The benchmark corpus has content hashes, datasheets, citation metadata, and loader checks. The operator confidence table is generated from code. The dashboard is regenerated from committed results. The public docs explain what the verifier can and cannot prove.

Reproducibility is paired with hygiene. Third-party source trees are quarantined or removed unless license-compatible. The security policy and threat model acknowledge that untrusted

model files are parsed and should not be executed dynamically. Safe loading paths use AST/fx-style extraction rather than arbitrary imports where possible. Baselines and suppressions let teams adopt the tool incrementally without hiding new regressions.

Limitations

TensorGuard is intentionally conservative. It is not a complete theorem prover for all Python or all PyTorch. It handles a documented verifiable fragment and falls back to UNKNOWN outside that fragment in strict modes. Data-dependent shape operators, arbitrary Python side effects, dynamic module mutation, runtime-generated code, and library internals that cannot be traced or modeled may require abstention. Symbolic arithmetic is limited to decidable fragments and practical solver budgets. Some integrations verify preconditions for export, compile, serving, quantization, or distributed execution rather than replacing those systems' own validators.

These limitations are design choices, not hidden caveats. The repository keeps an honest limitations document, a formal fragment specification, confidence tags for operators, and tests that assert unsupported constructs become UNKNOWN rather than SAFE. The most important invariant is not that every program is proved; it is that a strict safe verdict means what it says.

Why the Artifact Matters

For a tool to be credible at PLDI, OOPSLA, ISSTA, or a systems-for-ML venue, it must connect a formal idea to the messiness of real code. TensorGuard does that in three ways. First, it makes refinement-style tensor verification usable from normal PyTorch files with zero annotations. Second, it exposes enough product surface--CLI, CI, SARIF, Jupyter, decorators, framework hooks, export gates, Docker, conda, pipx, docs--that adoption can be evaluated rather than imagined. Third, it ships an artifact that reviewers can run, audit, and extend.

The codebase also provides a platform for future work. New operator rules can be added with confidence tags, conformance tests, and proof footprints. New benchmarks can enter the frozen corpus through provenance and hashing. New frontends can lower into the same graph representation. New proof-carrying certificates can reuse the existing safety certificate and replay infrastructure. The next natural milestones are larger blind real-bug corpora, broader deployment galleries, independently checkable certificates, and a community leaderboard that makes tensor-verifier progress measurable.

Conclusion

TensorGuard demonstrates that static verification for PyTorch tensor programs can be both principled and practical. The repository combines a refinement-type view of tensors, a multi-domain abstract state, Z3-backed checks, CEGAR refinement, soundness modes, precise operator transfer functions, real frontends, formal artifacts, and an unusually broad reproducibility harness. The latest chunk/split work illustrates the project's standard: match PyTorch's exact edge-case semantics, propagate precision to downstream model checks, abstain on symbolic uncertainty, and pin the behavior with live-library tests.

The result is a verifier that catches high-value architecture, device, dtype, phase, gradient, export, and deployment bugs before expensive training or opaque compiler failures. It is a research artifact with a clear trust boundary and a developer tool with practical adoption paths--the combination needed for a best-paper-quality static analysis system in modern ML software.

Capability Matrix

|p0.27|p0.64|

Capability and What the repository demonstrates

Zero-annotation source verification and Inference from constructors, layer definitions, tensor factories, asserts, shape aliases, and dataflow without requiring user type annotations.

Shape algebra and Matmul, reshape/view, flatten, broadcasting, cat, stack, chunk, split, transpose, permute, squeeze, unsqueeze, reductions, indexing, convolution, pooling, embedding, and attention rules.

Device reasoning and Propagation through tensor creation, module calls, `to`/`cuda`/ `cpu`, buffers, and cross-device operations.

Dtype reasoning and Promotion/cast checks, autocast boundaries, parameter/input dtype matching, quantization/export dtype gates, and dtype-related runtime-error prevention.

Phase reasoning and Train/eval tagging and phase-sensitive diagnostics for modules such as BatchNorm and Dropout.

Gradient reasoning and Detection of detached or broken gradient flow and training-loop checks around optimizers, parameters, and checkpoint-like patterns.

Stride and permutation and Layout and viewability support that keeps reshape/permute reasoning from collapsing to rank-only approximations.

Attention and SDPA and MHA contracts, q/k/v dimensions, kdim/vdim, masks, key padding masks, batch-first layout, and grouped head variants.

Sparse tensors and COO/CSR/CSC/BSR/BSC shape and blocksize invariants, with sparse operation extensions in progress.

Named tensors and `refine _names` and `align _to` alignment checks.

Complex and FFT and `view _as _real`, `view _as _complex`, FFT shape/dtype contracts.

Distributions and Batch shape, event shape, sample shape, and log-probability compatibility for probabilistic code.

vmap/functorch and Batched dimension transfer and out-dim contract checks.

Export and compile and Pre-pass verification for `torch.compile`, `torch.export`, ONNX, AOTInductor packaging, and deployment gates.

Distributed and adapters and DDP/FSDP/DTensor/tensor-parallel checks, optimizer-state checks, checkpoint schema checks, LoRA/PEFT adapter compatibility.

Reports and integrations and CLI, JSON, JUnit, SARIF, GitHub annotations, GitHub Action, pre-commit, pytest plugin, nox/tox, watch mode, Jupyter, decorators, and framework hooks.

Formal artifacts and Lean proof files, axiom audits, SMT encoding faithfulness checks, and proof certificate infrastructure.

Reproducibility and Frozen corpora, generated dashboards, numeric claim audits, benchmark scripts, hash manifests, datasheets, artifact badges, Docker/conda/source modes.

Benchline and Experiment Inventory

|p0.30|p0.61|

Artifact and Role in the evaluation story

`tests/test _split _chunk _precise.py` and Live PyTorch cross-checks and verifier regressions for exact chunk/split partition rules, empty chunks, negative dims, function/method forms, and downstream Linear/cat behavior.

`tests/test _mha _verify.py` and Multi-head attention conformance against real `nn.MultiheadAttention` and functional attention calls.

`tests/test _einsum _precise.py` and Equation parser and output-shape tests for einsum, including diagonals and ellipsis handling.

`tests/test _reshape _precise.py`, `tests/test _reshape _divisibility _opt.py` and Product reasoning, `-1` inference, and optimized divisibility constraints for reshapes/views.

`tests/test _broadcast _expand _precise.py`, `tests/test _broadcast _correspondence.py` and Broadcast and expand semantics, including singleton expansion and torch correspondence.

`tests/test _conv1d _rule.py`, `tests/test _conv2d _rule.py`, `tests/test _conv3d _rule.py`, `tests/test _pool2d _rule.py`, `tests/test _convtranspose _rule.py` and Convolution and pooling transfer functions over spatial arithmetic.

`tests/test _dtype _precise.py`, `tests/test _dtype _promote _chain.py`, `tests/test _bool _dtype _soundness.py` and Dtype propagation, promotion, and boolean/integer safety.

`tests/test _device _propagation _precise.py`, `tests/test _device _placement _transfer.py` and Device movement and cross-device error detection.

`tests/test _grad _flow _transfer.py`, `tests/test _training _loop _checks.py` and Gradient and training-loop contracts beyond forward shape checking.

`tests/test _onnx _export _gate.py`, `tests/test _export _aot _gate.py`, `tests/test _quant _export _checks.py` and Deployment-facing gates for export, opsets, AOT packaging, and quantization.

`tests/test _distributed _verification.py`, `tests/test _tensor _parallel _checks.py` and Distributed and tensor-parallel shape/device contracts.

`tests/test _lora _verification.py` and Adapter rank and target-module compatibility checks.

`tests/test _security.py`, `tests/test _threats _to _validity.py` and Security policy and evaluation-threat documentation.

`tests/test _docs _getting _started.py`, `tests/test _docs _limitations.py` and Executable documentation checks for onboarding and limitations.

`tests/test _paper _evidence _index.py`, `tests/test _paper _capsule _wiring.py` and Paper/artifact wiring so claims can be tied to scripts and committed evidence.

Transfer Rule Ledger

This appendix summarizes representative transfer rules as contracts rather than as an operator checklist. The purpose is to make clear which parts of the system are exact, which are sound but conservative, and which intentionally abstain.

|p0.22|p0.31|p0.36|

Family and Static contract and Evidence path

Tensor factories and `zeros`, `ones`, `rand`, `randn`, `empty`, `full`, and integer factories produce shapes determined by static size arguments, independent of random values. and Factory transfer tests and RNG shape tests; output shapes are propagated even when tensor contents are unknown.

Linear layers and The last input dimension must equal `in _features`; output replaces the last dimension with `out _features`. and Unit tests for clean and buggy linear models, autofix tests, and end-to-end module verification.

Matrix multiplication and Contracting dimensions must match; batched prefixes are propagated or broadcast where supported. and Shape arithmetic tests, Z3 symbolic checks, model-checker violations, and baseline bug cases.

Convolution and Input rank, channels, groups, kernel, stride, padding, dilation, and output padding determine channel and spatial output shapes. and Conv1d/2d/3d, transpose convolution, pooling, and pixel-shuffle regression tests.

Reshape/view and Element product is preserved; `-1` is inferred when uniquely determined; copied dimensions from `x.size(i)` and `x.shape[i]` retain aliases. and Precise reshape tests,

divisibility optimization tests, Lean-adjacent proof footprints, and real model regressions.

Flatten/unflatten and Flatten multiplies a contiguous dimension range when concrete and otherwise preserves rank with a symbolic product; unflatten checks product compatibility. and Flatten, unflatten, and reshape-chain tests.

Broadcast and expand and Right-aligned dimensions must be equal or singleton; expand may only grow singleton dimensions, and `-1` preserves an existing dimension where PyTorch allows it. and Broadcast correspondence, expand precision, and elementwise-operation tests.

Cat and All non-concatenation dimensions must match; concatenation dimension sums concrete sizes or becomes symbolic. Negative dims are normalized. and Lean cat rule, live PyTorch cat tests, analyzer and model-checker cat regressions.

Stack and All inputs must have equal rank and equal dimensions; output inserts a new axis of length equal to the number of tensors. and Stack tests and downstream shape propagation tests.

Chunk/split and Exact PyTorch partition sizes for concrete axes; concrete unpack-count errors; fresh-symbolic split axis when output count or size depends on symbols. and Live torch 2.9.1 cross-checks in `tests/test _split _chunk _precise.py` and downstream Linear/cat model checks.

Squeeze/unsqueeze and Squeeze removes singleton axes when statically known; unsqueeze inserts a singleton axis with negative-dim normalization. and Tensor-shape analyzer tests and model-checker step propagation.

Transpose/permute and Axis swaps and permutations reorder shape dimensions; identity permutes can be diagnosed as suspicious no-ops. and Transpose tests, permutation theory tests, identity-permute diagnostics.

Reductions and Dim reductions remove an axis or replace it with 1 under `keepdim`; global reductions produce scalar shapes. and Reduction tests for mean, sum, min, max, variance, standard deviation, and arg reductions.

Indexing and gather and Gather and take-along-dim return index shapes; index-select replaces the selected dimension; scatter preserves the input shape; narrow replaces a dimension with a known length. and Indexing/gather tests and extended operator tests.

Masked and value-dependent ops and Masked select, nonzero, one-argument where, and boolean masks check static mask contracts, then use symbolic selected lengths and explicit unknown reasons for value-dependent extents. and Boolean-mask tests against live PyTorch plus soundness-boundary tests for data-dependent ranks.

Einsum and Equation parser handles explicit and implicit outputs, ellipses, broadcasting, and repeated-label diagonal constraints where supported. and Einsum theory tests, precise einsum tests, and randomized torch differential batteries.

Attention and Q/K/V ranks, embedding dimensions, source lengths, batch layout, masks, and head-related contracts are checked for SDPA and MHA. and MHA verification tests against real PyTorch modules and functional calls; SDPA precise tests.

Normalization and BatchNorm, LayerNorm, GroupNorm, and InstanceNorm preserve shapes while checking channel or normalized-shape constraints when static. and Normalization rule tests and phase-flow tests.

Sparse layouts and Compressed and coordinate layouts carry index/value shape invariants and blocksize constraints. and Sparse verification tests across COO, CSR, CSC, BSR, and BSC.

Complex and FFT and Complex view conversions enforce final-dimension and dtype contracts; FFT families compute output dimensions such as real FFT half-spectrum sizes. and Complex verification tests, FFT tests, and operator confidence tags.

Named tensors and Name refinement and alignment require axis-name consistency and legal reorder operations. and Named tensor verification tests.

Distribution shapes and Batch, event, sample, and log-probability shapes must be mutually compatible. and Distribution verification tests and probabilistic-model examples.

vmap/functorch and Batched dimensions are inserted, removed, or moved according to `in _dims` and `out _dims`; impossible batch contracts are rejected. and vmap verification tests and live dispatcher checks.

Export gates and Export, ONNX, AOTInductor, and quantization paths check static preconditions before invoking downstream compilers. and ONNX, AOT, quantization, and guarded compile/export tests.

Distributed and adapter gates and Sharding, tensor parallelism, optimizer state, checkpoints, and LoRA adapters have shape/dtype/rank contracts layered over base verification. and Distributed verification, tensor-parallel, training-loop, checkpoint, and LoRA tests.

Experiment Protocol Notes

The empirical suite is intentionally redundant. Each protocol checks a different failure mode in the claim stack, and overlap is useful: a bug caught by a real-bug corpus, a mutation benchmark, and a live torch conformance test is less likely to be an artifact of one benchmark's construction.

Real-bug protocol

Real bugs are valuable only when provenance is preserved. The repository's corpus entries carry labels, hashes, and loader checks so that a benchmark case cannot silently drift. GitHub-mined cases are derived from public issue and PR signals, especially runtime error messages that indicate shape or device failures. The intent is not to treat every mined candidate as a theorem; it is to create a large, auditable pool that can be stratified, manually labeled, filtered into blind splits, and reused by future tools.

Clean-model protocol

False positives are more damaging than missed bugs for a CI gate. Clean-model tests therefore exercise patterns that static tools often mishandle: symbolic batch sizes, shape aliases, helper functions, container modules, attention wrappers, non-divisible but valid partitions, singleton broadcasting, empty sections, and deployment preconditions that should abstain rather than refute. The strict soundness mode is evaluated on these clean cases to maintain the promise that a trusted safe verdict is not an accident of permissive defaults.

Differential protocol

For operator semantics, PyTorch is the executable specification. Differential tests generate tensors, run PyTorch, and compare the observed shape or exception to TensorGuard's transfer function. This protocol is especially useful for edge cases whose behavior is not obvious from memory: `torch.chunk` returning fewer than requested chunks, zero-size split sections, broadcasting of attention masks, compressed sparse block constraints, and named-axis alignment.

Mutation protocol

Mutation tests start from clean models and introduce controlled faults: wrong feature counts, wrong channels, bad reshape products, device moves, dtype casts, gradient detaches, invalid sparse metadata, and export-incompatible layouts. A mutation is useful only if it represents a runtime or semantic failure that real code could contain. The harness therefore records the mutant, the expected outcome, and the evidence that the mutation is admitted by the source syntax but rejected by the verifier or runtime.

Ablation protocol

Ablations isolate the contribution of each abstract domain and each refinement component. Shape-only, device-only, dtype-only, gradient-only, independent domains, reduced products, CEGAR depths, operator coverage levels, solver timeouts, and frontend variants can be compared. This is the right protocol for answering whether the tool's complexity is justified. If a domain never catches a unique bug, it should either be simplified or documented as diagnostic.

Latency protocol

Static verification must run before it can help. The latency protocol measures small, medium, and large models; import-time, install-time, and analysis-time costs; solver calls avoided by syntactic transfer functions; incremental reverification under diffs; parallel per-module verification; and timeout behavior. The important metric is not only mean runtime but the tail behavior under symbolic constraints and unsupported regions.

|p0.25|p0.31|p0.33|

Protocol and Primary risk controlled and Representative assertion

Real-bug corpus and Overfitting to toy bugs and Frozen labels and hashes reproduce expected verdicts.

Clean-model stress and CI-hostile false positives and Strict mode does not refute valid real-model patterns.

Differential torch oracle and Incorrect operator semantics and Static transfer matches PyTorch shape or exception on generated cases.

Mutation benchmark and Weak recall on realistic edits and Injected faults are detected at the intended operation site.

Fuzzing and Hidden disagreement surfaces and Minimized disagreements become regression tests.

Baseline comparison and Unclear value over existing tools and Precision, recall, latency, and time-to-detect are compared against external alternatives.

Domain ablation and Unjustified abstract domains and Removing a domain changes measurable recall, precision, diagnostics, or runtime.

CEGAR depth ablation and Refinement over- or under-use and More refinement improves specific cases until convergence without unbounded runtime.

Operator coverage gate and Silent regression in transfer surface and Coverage and confidence tables remain in sync with code.

Dashboard regeneration and Stale paper claims and Committed result artifacts match freshly regenerated outputs.

Statistical checks and Underpowered headline claims and Effect-size and sample-size artifacts document uncertainty.

Artifact packaging and Reproducibility friction and Docker, conda, source, and pipx paths can run the advertised checks.

Adoption Scenarios

Local model authoring

A researcher writing a new module can run `tensorguard verify model.py` before starting training. The verifier infers shapes from common constructors, input hints, docstrings, and structural layer usage. If it finds a bad `Linear` feature count or attention mask, it reports a source-mapped diagnostic. If it reaches unsupported dynamic behavior, strict mode reports UNKNOWN with an explanation. This keeps the feedback loop close to the edit, not to the first expensive batch.

Continuous integration

In CI, teams can run high-confidence or sound mode on changed model files. The GitHub Action can annotate pull requests and SARIF can surface findings in code scanning. Baseline/suppression support lets a legacy repository adopt the tool incrementally: existing issues can be acknowledged while new regressions fail the build. Domain flags make ablation and staged rollout practical; for example, a team may initially enforce shape and dtype while logging phase diagnostics.

Notebook exploration

Jupyter/IPython integration checks a model cell when it is defined. This mode is intentionally lightweight. A notebook user benefits from early shape and device feedback without building a package or writing a config file. Because notebooks often contain dynamic or partially executed state, honest abstention is important: the tool should explain what it could not prove rather than pretend that notebook context is a static module.

Framework integration

Framework authors can call the public API before launching training, compiling a model, exporting an artifact, or serving a checkpoint. Hooks for Lightning, HuggingFace-style trainers, accelerate-like launchers, MLflow, W and B, and CI systems allow TensorGuard to act as a precondition gate. The value is not merely a shape checker; it is a common safety layer that can speak JSON, SARIF, JUnit, or native annotations.

Deployment gate

Deployment introduces contracts that ordinary training does not: ONNX opset availability, AOTInductor input layouts, quantization observer placement, mixed-precision dtype boundaries, TorchServe or FastAPI request schemas, GGUF or llama.cpp checkpoint shapes, CUDA graph capture assumptions, and distributed shard boundaries. TensorGuard's deployment modules check these contracts before the downstream system emits a harder-to-debug compiler, exporter, or serving error.

Research artifact review

A reviewer can start from the soundness contract, run the new tests for any claimed operator family, inspect the evidence index, regenerate dashboards, and audit Lean files. This is a different posture from a one-off prototype: the paper claims are connected to code paths, tests, and artifacts, and unsupported regions are part of the documented result.

Design Decisions Worth Preserving

Abstention is a feature. A static tensor verifier that guesses past unknown code can appear impressive on demos while being unusable in CI. TensorGuard instead treats UNKNOWN as a first-class outcome. This enables strict adoption: a user can decide whether an unknown is acceptable for their workflow.

Concrete bugs should be concrete. When a shape is known, the diagnostic should name the operation and the mismatched dimension. ``Linear expects last dim=4, got 2" is more useful than ``solver found SAT." Counterexamples are still valuable, but they should support the explanation rather than replace it.

Operator rules must match the live library. PyTorch changes, and some operator behavior is subtle. Transfer functions should be cross-checked against the installed PyTorch version wherever possible. The chunk/split rule is a model example: intuitive equal partitioning would be wrong, so the rule follows observed torch behavior.

Integration surfaces are part of correctness. A verifier that catches bugs only through an internal Python function is not enough. CI, SARIF, pre-commit, notebooks, watch mode, export gates, and framework hooks determine whether the analysis affects real development.

Evidence should regenerate. The repository's JSON artifacts, dashboards, coverage tables, and paper evidence files exist so claims can be checked. A best-paper-grade artifact must make it easy to see when a number or guarantee is stale.

Formalization should target the trust core. Proving all of PyTorch is not the goal. Proving the algebra that the implementation relies on, auditing axioms, and pairing proofs with conformance tests creates a tractable and useful trust story.

Diagnostic Taxonomy

The user-facing product of the verifier is a diagnostic, not a theorem object. The following taxonomy keeps diagnostics comparable across domains and output formats.

|p0.24|p0.31|p0.34|

Diagnostic class and Typical trigger and Useful response

Dimension mismatch and A contract expects two static dimensions to be equal and they are not. and Report both dimensions, the axis, and the operation that imposed the equality.

Rank mismatch and An operator requires a tensor rank or rank range that the input does not satisfy. and Report expected rank, observed rank, and whether a reshape, unsqueeze, or batch dimension is likely involved.

Product mismatch and A reshape, view, flatten, or unflatten changes the number of elements. and Report the source product, target product, and the unresolved symbolic factors if any.

Partition mismatch and Split, chunk, unbind, or unpacking returns a different number of values than the program expects. and For concrete cases, report the exact returned count; for symbolic cases, abstain with a dependency explanation.

Broadcast failure and Elementwise or mask shapes cannot be right-aligned under singleton expansion. and Report the first incompatible aligned axis and both input shapes.

Device conflict and Inputs to an operation reside on incompatible devices or an implicit host/device transfer would violate the selected policy. and Report the producer operation for each device and any missing explicit move.

Dtype conflict and Mixed dtypes are unsupported, lossy, or inconsistent with an operator's contract. and Report promoted or required dtype and the tensor whose dtype blocks the operation.

Gradient break and A training path detaches, casts, mutates, or guards tensors in a way that prevents intended gradient flow. and Report the first operation that breaks differentiability and the downstream loss or parameter it affects.

Export precondition and ONNX, AOT, quantization, serving, or accelerator-specific constraints are violated. and Report the backend-specific contract and the nearest portable replacement if known.

Unsupported dynamic region and The program constructs shapes or control flow from values that the selected analysis mode cannot resolve. and Return UNKNOWN with the unsupported expression and a suggestion for an input hint or annotation.

Baseline-suppressed issue and A finding exists but is acknowledged by an adoption baseline. and Keep the evidence in machine-readable output while not failing the incremental gate.

Evidence inconsistency and A dashboard, paper artifact, corpus label, or coverage table is stale relative to code or tests. and Fail regeneration checks and point to the artifact that must be rebuilt.

Reviewer Checklist

A reviewer can inspect the trust boundary by reading `SOUNDNESS _CONTRACT.md`, `VERIFIABLE _FRAGMENT.md`, and `operator _confidence _table.json`. They can run the new partition tests with: [`python -m pytest -q tests/test _split _chunk _precise.py`.] They can inspect the empirical story through `make reproduce`, the dashboard tests, real benchmark loaders, baseline comparisons, and the reproducibility directory. They can inspect formal claims through the Lean files and axiom-audit tests. Most importantly, they can observe that unsupported or symbolic-hard cases are not hidden: TensorGuard either proves, refutes, or says UNKNOWN with a reason.